ROTEIRO PRÁTICO DA DISCIPLINA

Curso *Engenharia de Software*	**Campus** *Rio Verde*
Disciplina *Programação Orientada a Objetos*	
Professor *Sergio Souza Novak*	**Turma** *A / B*

Tratamento de Exceções em Java

Um ótimo jeito de aprender exceções em Java é modelar situações reais de uma conta corrente, porque vários erros fazem sentido no domínio do problema:

- saldo insuficiente
- valor inválido
- conta inexistente
- saque acima do limite
- transferência para a mesma conta

Isso ajuda a entender:

- `throw`
- `throws`
- exceções personalizadas
- `try/catch`
- checked vs unchecked exceptions

Roteiro Prático

1. Criar a classe da conta

```
public class ContaCorrente {

    private String titular;
    private double saldo;

    public ContaCorrente(String titular, double saldoInicial) {
        this.titular = titular;
        this.saldo = saldoInicial;
    }

    public double getSaldo() {
        return saldo;
    }
}
```

2. Criar uma exceção personalizada

Exemplo: saldo insuficiente

```
public class SaldoInsuficienteException extends Exception {

    public SaldoInsuficienteException(String mensagem) {
        super(mensagem);
    }
}
```

Aqui você aprende:

- herança
- `extends Exception`
- construtor de exceção

3. Lançar exceção com `throw`

Agora vamos criar o saque.

```
public void sacar(double valor)
        throws SaldoInsuficienteException {

    if (valor > saldo) {
        throw new SaldoInsuficienteException(
                "Saldo insuficiente para saque."
        );
    }

    saldo -= valor;
}
```

4. Tratar a exceção com `try/catch`

```
public class Main {

    public static void main(String[] args) {

        ContaCorrente conta =
                new ContaCorrente("Ana", 500);

        try {

            conta.sacar(1000);

        } catch (SaldoInsuficienteException e) {

            System.out.println("Erro: " + e.getMessage());

        }

        System.out.println("Programa continua...");
    }
}
```

Fluxo do que acontece

```
main()
  ↓
sacar()
  ↓
throw new SaldoInsuficienteException()
  ↓
catch()
```

5. Criar mais exceções

Valor inválido

```
public class ValorInvalidoException
        extends RuntimeException {

    public ValorInvalidoException(String mensagem) {
        super(mensagem);
    }
}
```

6. Trabalhar checked vs unchecked

Checked Exception

- `extends Exception`
- Java obriga: `try/catch` ou `throws`

Unchecked Exception

- `extends RuntimeException`
- Java NÃO obriga tratamento.

Exemplo prático

```
public void depositar(double valor) {

    if (valor <= 0) {
        throw new ValorInvalidoException(
                "Valor inválido."
        );
    }

    saldo += valor;
}
```

7. Transferência entre contas

Excelente exercício para praticar propagação de exceções.

```
public void transferir(
        ContaCorrente destino,
        double valor)
        throws SaldoInsuficienteException {

    this.sacar(valor);

    destino.depositar(valor);
}
```

8. Exceções Nativas Comuns no Java

Além das exceções personalizadas, a linguagem Java possui várias exceções nativas que ocorrem frequentemente no dia a dia. Você não precisa criá-las, mas é fundamental saber como evitá-las ou tratá-las.

- `NullPointerException`: Ocorre quando você tenta acessar um método ou atributo de um objeto que está com referência nula.

```
ContaCorrente conta = null;
conta.getSaldo(); // Lança NullPointerException
```

- `IllegalArgumentException`: Usada quando um método recebe um argumento inadequado. Em vez de criar a `ValorInvalidoException`, muitas vezes podemos usar esta exceção nativa para validações:

```
public void depositar(double valor) {
    if (valor <= 0) {
        throw new IllegalArgumentException("Valor inválido.");
    }
    saldo += valor;
}
```

- `ArithmeticException`: Erros de matemática, como divisão por zero.

```
int resultado = 10 / 0; // Lança ArithmeticException
```

- `ArrayIndexOutOfBoundsException`: Tentativa de acessar uma posição que não existe em um array.

```
int[] numeros = {1, 2, 3};
System.out.println(numeros[5]); // Lança ArrayIndexOutOfBoundsException
```

9. Exercícios recomendados

- **Básico:** saque simples, depósito, saldo insuficiente.
- **Intermediário:** limite diário, conta bloqueada, valor negativo.
- **Avançado:** transferência, logs de erro, hierarquia de exceções, múltiplos catches, `finally`.

Exemplo com `finally`

```
try {
    conta.sacar(1000);
} catch (SaldoInsuficienteException e) {
    System.out.println(e.getMessage());
} finally {
    System.out.println("Operação encerrada.");
}
```

Estrutura ideal do projeto

```
src/
 |-- Main.java
 |-- ContaCorrente.java
 \-- exceptions/
      |-- SaldoInsuficienteException.java
      \-- ValorInvalidoException.java
```

Conceitos que você aprende nesse exemplo

Conceito	Onde aparece
`throw`	lançar exceção
`throws`	propagar exceção
`try/catch`	tratamento
`finally`	encerramento
Herança	exceções personalizadas
Polimorfismo	RuntimeException vs Exception
Encapsulamento	saldo privado
Regras de negócio	validações bancárias